

Angular 3+ Years Experience

100 Comprehensive Technical Interview Questions & Mastery Checklist

ANGULAR V17/V18+

SIGNALS & MODERN CONTROL FLOW

RXJS & ARCHITECTURE

Target Role: Mid-to-Senior Angular Developer (3+ Years Experience)

Focus Areas: Architecture, Signals, RxJS Reactive Streams, Performance Optimization, Dependency Injection, Standalone Architecture, Dynamic Forms & Security.

1. Core Concepts & Architecture

Questions 1 – 15

1. What are **Standalone Components**, and how do they replace traditional **@NgModule**s?
2. Explain the core differences between **AngularJS (v1)** and modern **Angular (v2+)**.
3. What is the **Angular CLI**, and what are custom CLI schematics?
4. Explain the difference between **declarations**, **imports**, **providers**, and **exports** in Angular modules.
5. How does Angular handle **DOM Sanitization** to prevent Cross-Site Scripting (XSS) attacks?
6. What are **Angular Decorators** (**@Component**, **@Directive**, **@Pipe**, **@Injectable**) and how do they work under the hood?
7. What is the role of **main.ts** and **angular.json** in bootstrapping an application?
8. Explain the new **Built-in Control Flow** (**@if**, **@for**, **@switch**) vs. traditional structural directives (***ngIf**, ***ngFor**).
9. Why is the **track** expression mandatory in the new **@for** block, and how does it compare to **trackBy**?
10. What is **ViewEncapsulation**? Compare **Emulated**, **None**, and **ShadowDom**.
11. What is the difference between a **Component** and a **Directive**?
12. What are **Attribute Directives** vs **Structural Directives**? Give real-world examples.
13. How do you construct custom directives to modify element behaviors?
14. What are **Pipes**? Differentiate between Pure and Impure pipes with practical examples.
15. How does the **async** pipe work, and why does it prevent memory leaks?

2. Components, Signals & Reactive Primitives

Questions 16 – 30

1. What are **Angular Signals** (`signal()` , `computed()` , `effect()`), and why were they introduced?
2. How do Signals improve Angular's reactivity compared to RxJS Observables?
3. What is the difference between `computed()` and `effect()` ? When should you *avoid* using `effect()` ?
4. How do Signal Inputs (`input()` , `input.required()`) replace `@Input()` decorators?
5. What are Signal Outputs (`output()`) and Model Inputs (`model()`)?
6. What are **ViewChild**, **ViewChildren**, **ContentChild**, and **ContentChildren** (and their Signal counterparts `viewChild` , etc.)?
7. Explain **Content Projection** and how multi-slot content projection works using `<ng-content select="...">` .
8. What is the difference between `ng-template` , `ng-container` , and `ng-content` ?
9. Explain the Component Lifecycle hooks in order (`ngOnChanges` , `ngOnInit` , `ngDoCheck` , etc.).
10. How do lifecycle hooks change when switching to full Signal-based components?
11. What is `DestroyRef` and `takeUntilDestroyed` ? How do they replace `ngOnDestroy` ?
12. How does parent-child component communication work without RxJS?
13. How do you communicate between non-related (sibling) components?
14. What is **Dynamic Component Loading** and when would you use `ViewContainerRef.createComponent()` ?
15. What are `afterRender` and `afterNextRender` lifecycle callbacks?

3. Dependency Injection (DI) & Services

Questions 31 – 42

1. How does **Dependency Injection** work in Angular?
2. What is the difference between `providedIn: 'root'` , `providedIn: 'platform'` , and module/component providers?
3. Explain the **Hierarchical Dependency Injection** system.
4. What is the difference between `providers` and `viewProviders` in `@Component` metadata?
5. How does the `inject()` function work, and why is it preferred over constructor injection?
6. What is an `InjectionToken` and when should you create custom ones?
7. What are Resolution Modifiers (`@Optional()` , `@Self()` , `@SkipSelf()` , `@Host()`)?
8. What are `useClass` , `useValue` , `useFactory` , and `useExisting` provider configurations?
9. How do you write a factory provider that depends on another asynchronous service?
10. How do singleton services behave in lazy-loaded modules vs root injectors?
11. What happens if you provide a service in a component's `providers` array instead of `@Injectable({ providedIn: 'root' })` ?
12. How does Angular handle circular dependency injection issues?

4. RxJS & Asynchronous Programming

Questions 43 – 57

1. What is the fundamental difference between an **Observable** and a **Promise**?
2. What is the difference between **Cold** and **Hot** Observables?
3. Explain the difference between **Subject**, **BehaviorSubject**, **ReplaySubject**, and **AsyncSubject**.
4. Explain higher-order mapping operators: **switchMap**, **mergeMap**, **concatMap**, and **exhaustMap**.
5. When would you choose **exhaustMap** over **switchMap**?
6. What is the difference between **share()** and **shareReplay()**?
7. How do you combine multiple observables using **combineLatest**, **forkJoin**, **zip**, and **merge**?
8. What is the difference between **forkJoin** and **combineLatest**?
9. How do you handle RxJS error handling using **catchError**, **retry**, and **retryWhen**?
10. What are memory leaks in RxJS and how do you prevent them?
11. How does **toSignal()** and **toObservable()** bridge RxJS and Signals?
12. What is the RxJS "Diamond Problem" and why does Signal architecture resolve it?
13. How do you debug complex RxJS stream pipelines?
14. What is the **tap** operator and what are its best practices?
15. Explain backpressure strategy handling in RxJS (e.g., **debounceTime** vs **throttleTime**).

5. Angular Forms

Questions 58 – 68

1. Compare **Template-Driven Forms** and **Reactive Forms**.
2. What are **FormControl**, **FormGroup**, **FormArray**, and **FormRecord**?
3. How do you build dynamic forms with **FormArray**?
4. How do you implement custom sync and async validators in Reactive Forms?
5. What is **ControlValueAccessor** (CVA) and how do you build custom form controls?
6. How do **valueChanges** and **statusChanges** observables work in Angular forms?
7. How do you update form values using **patchValue()** vs **setValue()**?
8. What are the modern typed forms introduced in Angular, and what problem do they solve?
9. How do you trigger validation programmatically across an entire form on submit?
10. How do you conditionally change validation rules based on user input?
11. How do you handle cross-field form validation (e.g., password matching)?

6. Routing, Navigation & Performance

Questions 69 – 82

1. How do you configure Routing in Angular without `AppRoutingModule` (Standalone setup)?
2. What are **Route Guards** (`canActivate` , `canDeactivate` , `canMatch`) and how are functional guards written?
3. What is **Lazy Loading** and how do you implement it for Standalone Routes?
4. How does route preloading strategy work (`PreloadAllModules` vs Custom Preloading)?
5. What is the `ActivatedRoute` snapshot vs `ActivatedRoute` observable streams?
6. How do **Resolvers** (`resolve`) work in modern Angular routing?
7. What is the `ComponentInputBinding` feature in Angular Routing (`withComponentInputBinding`)?
8. What is Angular's default **Change Detection Strategy** (`Default` vs `OnPush`)?
9. How does `OnPush` change detection work, and what triggers a view update under it?
10. What is `Zone.js` and how does Angular know when to run change detection?
11. What is **Zoneless Angular**, and how do Signals enable Zoneless applications?
12. How do you manually trigger change detection using `ChangeDetectorRef` (`markForCheck()` , `detectChanges()`)?
13. What is **Deferrable Views** (`@defer`) and how does it optimize initial bundle loads?
14. Explain `@defer` triggers (`on viewport` , `on interaction` , `on hover` , `when condition` , `prefetch`).

7. HTTP, Security & Server-Side Rendering

Questions 83 – 92

1. How do HTTP Interceptors work in Angular using `provideHttpClient(withInterceptors([...]))` ?
2. How do you handle authentication tokens (JWT) using HttpInterceptors?
3. How do you handle global error logging with HTTP Interceptors?
4. What is **Angular Universal** / SSR (Server-Side Rendering) and Hydration?
5. What is **Non-destructive Hydration** in Angular SSR?
6. How do you handle browser-only APIs (`window` , `document`) when running SSR applications?
7. How does Angular protect against **CSRF / XSRF** attacks?
8. What is `DomSanitizer` and when should you use `bypassSecurityTrustHtml` ?
9. How do you implement CORS policies handling in Angular client apps?
10. What are Progress Web Apps (PWA) and Service Workers in Angular?

8. State Management, Testing & Optimization

Questions 93 – 100

1. Compare component-level state management (Signals/Services) vs Global state (NgRx / ComponentStore).
2. What are the core building blocks of **NgRx** (**Store** , **Actions** , **Reducers** , **Selectors** , **Effects**)?
3. What is the **NgRx Signal Store** and how does it differ from traditional Redux NgRx?
4. How do you configure **TestBed** for unit testing Angular components and services?
5. How do you mock services and HTTP calls during unit testing (**HttpTestingController**)?
6. What is the difference between **fakeAsync** + **tick()** vs **async** / **waitForAsync()** in Angular unit tests?
7. How do you analyze and optimize Angular bundle sizes using source-map-explorer or webpack bundle analyzer?
8. How do you optimize web vital metrics (LCP, CLS, FID/INP) in an Angular application?